

2NF
 A relation is in 2NF if:
 1. It is already in 1NF
 2. It has no partial dependency
 3. Every non-prime attribute must depend on the whole candidate key, not just part of it

Main rule of 2NF
 If the key is composite, then:
 • CK $\rightarrow$ NP is accepted
 • Part of CK $\rightarrow$ NP is not accepted

3NF
 A relation is in 3NF if:
 1. It is already in 2NF
 2. It has no transitive dependency
 3. For every functional dependency $X \rightarrow Y$, A, at least one of these must be true:
 • X is a superkey
 • A is a prime attribute

Main rule of 3NF
 • If left side is not a superkey, then right side must be prime to be allowed.

BCNF
 A relation is in BCNF if:
 For every functional dependency $X \rightarrow Y$:
 • X must be a superkey

Main rule of BCNF
 • Every determinant must be a superkey
 BCNF is stricter than 3NF.

2) Very important shortcut table
 This is the part that usually makes everything clear.

Dependency type	2NF	3NF	BCNF	Comment
CK $\rightarrow$ NP	☑ Allowed	☑ Allowed	☑ Allowed	Good dependency

Prime and non-prime attributes

Attribute	Type
A	Prime
B	Prime
C	Prime
D	Non-prime

5) Check which normal form is violated

Check 2NF

FD: $C \rightarrow D$

Here:
 • C is part of candidate key AC
 • C is part of candidate key BC
 • D is non-prime

So this is a partial dependency.

Therefore:
 • 2NF is violated

Check 3NF

FD: $C \rightarrow D$

• C is not a superkey
 • D is non-prime

So this violates 3NF too.

FDs:
 • A is not a superkey by itself
 • B is prime

Dependency type	2NF	3NF	BCNF	Comment
CK $\rightarrow$ P	☑ Allowed	☑ Allowed	☑ Allowed	Good dependency
Part of CK $\rightarrow$ NP	☒ Not allowed	☒ Not allowed	☒ Not allowed	This is partial dependency
P $\rightarrow$ NP	☒ Usually not allowed if it causes partial dependency	☒ Not allowed unless P is superkey	☒ Not allowed unless P is superkey	Prime on left does not automatically make it valid
P $\rightarrow$ P	☑ ☒ depends	☑ Allowed if RHS is prime	☒ Not allowed unless P is superkey	3NF may allow it
NP $\rightarrow$ P	☑ ☒ depends	☑ Allowed if both is prime	☒ Not allowed unless P is superkey	Allowed in 3NF only if RHS is prime
NP $\rightarrow$ NP	☑ ☒ depends	☒ Not allowed unless NP is superkey	☒ Not allowed unless NP is superkey	Usually bad

3) Easy memory rule

For 2NF

- Focus on partial dependency
- A non-prime attribute must not depend on part of a composite key

For 3NF

- Focus on transitive dependency
- X $\rightarrow$ A is okay if:
 - X is a superkey, or
 - A is prime

For BCNF

- Focus on determinant
- X must always be a superkey

4) Tricky problem with solution

- A is prime

So these are allowed in 3NF but not allowed in BCNF

Therefore:

- The relation is not in 3NF because of C $\rightarrow$ D
- Even after fixing that part, A $\rightarrow$ B and B $\rightarrow$ A will still break BCNF

6) Decomposition step by step

First decomposition for 2NF

Split off C $\rightarrow$ D

Relations:

Relation	Attributes
R1	(C, B)
R2	(A, B, C)

Check R2(A, B, C)

FDs in R2: $A \rightarrow B$
 $B \rightarrow A$

Candidate keys in R2:

- AC
- BC
- A, B, C

Normal form of R2

- It is in 3NF
- It is not in BCNF

Why? Because:

- A $\rightarrow$ B has determinant A, which is not a superkey
- B $\rightarrow$ A has determinant B, which is not a superkey

But both RHS attributes are prime, so 3NF is okay.

7) Final BCNF decomposition

To make R(A, B, C) into BCNF split based on:

$A \rightarrow B$

New relations:	
Relation	Attributes
R1	(A, B)
R2	(A, C)

Now the full final decomposition is:

Final Relation	
R1	(C, D)
R3	(A, B)
R4	(A, C)

8) Final answer summary

Step	Result
Original relation	R(A, B, C, D)
Candidate keys	AC, BC
Prime attributes	A, B, C
Non-prime attribute	D
2NF violation	C → D
3NF violation	C → D
BCNF violation	A → B, B → A, C → D
Final BCNF tables	R1(C,D), R3(A,B), R4(A,C)

9) Exam-ready one-line rules

3) Which decomposition method is easiest

Easiest method for 3NF

The easiest and safest method is the 3NF synthesis method.

Why it is easy

- It guarantees lossless join
- It usually preserves dependencies
- It gives a clean decomposition from functional dependencies directly

Basic idea

- Find the minimal cover of FDs
- Make one relation for each FD
- If one relation contains a candidate key, you are done
- Remove redundant relations

BCNF decomposition is stricter

BCNF gives a cleaner structure, but it may not preserve all dependencies.

Why BCNF is harder

- Every determinant must be a superkey
- If not, split again
- After splitting, some dependencies may be lost

So		
Normal form target	Ease	Dependency preservation
3NF	Easier	Usually preserved
BCNF	Harder	Not always preserved

4) What to check first in every problem

Before solving any normalization problem, check these things in this order:

Step	What to check
1	Functional dependencies
2	Candidate keys
3	Prime and nonprime attributes
4	Partial dependency
5	Transitive dependency

Normal Form	One-line rule
2NF	No partial dependency of non-prime attributes on part of a candidate key
3NF	No transitive dependency for $X \rightarrow Y$, either X is superkey or Y is prime
BCNF	For every FD $X \rightarrow Y$, X must be a superkey

If you need it, I can turn this into a one-page revision sheet with even simpler tables and a second example using 5 attributes.



Let problems like above in this way more complex around 3 problems and also list explain the types of the decomposition and what kind of the decompositions method is easy to resolve and what are the rules that we need to follow to decompose into as a simple method as possible and also include the rules that primarily need to be checked mainly is that I want to know the detailed view of how does it actually look at the problem and describe so show me three such problems and resolve as per the question like if asked for resolve bc only for bc and then if for the 3 then 3nf in this way explain me in depth. dont include emojis

Thought for a couple of seconds

Below is a neat, exam-friendly guide to help you understand normalization and decomposition deeply, with three worked problems.

1) What decomposition means in DBMS

A relation is decomposed when we split one table into smaller tables to reduce redundancy and anomalies.

Example	Original relation	Decomposed into
	R(A, B, C, D)	R1(A, B) and R2(A, C, D)

Step	What to check
6	Whether determinant is a superkey
7	Whether bodies join is preserved
8	Whether dependencies are preserved

5) Main rules to remember

For 2NF

- The relation must be in 1NF
- No partial dependency
- Non-prime attributes should depend on the whole candidate key

2NF violation pattern

If the key is composite and

- Part of $CK \rightarrow NP$ then it violates 2NF

For 3NF

For each functional dependency $X \rightarrow Y$, at least one must hold:

- X is a superkey
- Y is a prime attribute

3NF violation pattern

- Non-superkey determines a non-prime attribute

For BCNF

For each functional dependency $X \rightarrow Y$:

- X must be a superkey
- Any FD where the left side is not a superkey

6) Important dependency patterns

This is a very useful exam shortcut.

The goal is to make the design:

Goal	Meaning
Lossless join	No information is lost after splitting
Dependency preservation	Functional dependencies can still be enforced without joining tables
Better normal form	Remove redundancy and anomalies

2) Main types of decomposition

A. Lossless-join decomposition

A decomposition is lossless if we can join the smaller relations and get back the original relation exactly.

Simple rule to check lossless join

For decomposition into R1 and R2:

If

$R1 \cap R2 \rightarrow R1$

or

$R1 \cap R2 \rightarrow R2$

then the decomposition is lossless.

In simple words:

the common attributes should functionally determine one of the relations.

B. Lossy decomposition

If joining the decomposed tables produces extra rows or incorrect results, the decomposition is lossy.

This is bad.

C. Dependency-preserving decomposition

A decomposition is dependency preserving if all functional dependencies can be checked in the smaller tables without joining them.

This is very important in 3NF design.

FD type	2NF	3NF	BCNF
$CK \rightarrow NP$	Allowed	Allowed	Allowed
$CK \rightarrow P$	Allowed	Allowed	Allowed
Part of $CK \rightarrow NP$	Not allowed	Not allowed	Not allowed
$P \rightarrow NP$	Usually not allowed if it causes partial dependency	Not allowed unless P is superkey	Not allowed unless P is superkey
$P \rightarrow P$	May be allowed	Allowed if RHS is prime	Not allowed unless P is superkey
$NP \rightarrow P$	May be allowed in some cases	Allowed if RHS is prime	Not allowed unless NP is superkey
$NP \rightarrow NP$	Usually bad	Not allowed unless NP is superkey	Not allowed unless NP is superkey

Here:

- P = prime attribute
- NP = non-prime attribute
- CK = candidate key

7) How to solve decomposition problems step by step

For 2NF problems

- Find candidate key
- Check whether the key is composite
- Find partial dependencies
- Split off attributes that depend on part of the key

For 3NF problems

- Find candidate keys
- Identify prime attributes
- Look for transitive dependencies
- Decompose so that no non-prime attribute depends on another non-key attribute

For BCNF problems

- Find all FDs
- Check whether each determinant is a superkey
- If not, decompose using that FD

4. Repeat until every FD has a superkey on the left

8) Three solved problems

I will now give three increasingly useful examples:

- Problem 1: 2NF-oriented
- Problem 2: 3NF-oriented
- Problem 3: BCNF-oriented

Problem 1: 2NF decomposition

Relation

$R(A, B, C, D)$

Functional dependencies

$AB \rightarrow C$
 $A \rightarrow D$

Step 1: Find candidate key

Check as:

- $AB \rightarrow C$
- $A \rightarrow D$

So AB determines all attributes.

Thus:

AB

is the candidate key.

Step 2: Prime and non-prime attributes

Attribute	Type
A	Prime
B	Prime
C	Non-prime
D	Non-prime

FD 4: $D \rightarrow E$

- D is not a superkey
- E is non-prime
- Violates 3NF

So the relation is not in 3NF.

Step 4: Decompose using 3NF synthesis idea

Create relations from FDs:

FD	Relation
$A \rightarrow B$	(A, B)
$B \rightarrow C$	(B, C)
$AC \rightarrow D$	(A, C, D)
$D \rightarrow E$	(D, E)

Now check whether a relation contains a candidate key.

Candidate key is A .

Relation (A, B) contains A , so key is present.

So final decomposition is:

Relation	Attributes
R_1	(A, B)
R_2	(B, C)
R_3	(A, C, D)
R_4	(D, E)

Final answer for Problem 2

Result:	Value
Candidate key	A
3NF violations	$B \rightarrow C, D \rightarrow E$
Final 3NF decomposition	$R_1(A, B), R_2(B, C), R_3(A, C, D), R_4(D, E)$

Step 3: Check for 2NF violation

FD:

$A \rightarrow D$

Here:

- A is part of candidate key AB
- D is non-prime

So this is a partial dependency.

Thus the relation is not in 2NF.

Step 4: Decompose

Split as:

Relation	Attributes	Reason
R_1	(A, B)	because $A \rightarrow B$
R_2	(A, B, C)	remaining part

Final answer for Problem 1

Result:	Value
Candidate key	AB
2NF violation	$A \rightarrow D$
Decomposition	$R_1(A, B), R_2(A, B, C)$

Problem 2: 3NF decomposition

Relation

$R(A, B, C, D, E)$

Functional dependencies

$A \rightarrow B$
 $B \rightarrow C$
 $AC \rightarrow D$

Problem 3: BCNF decomposition

Relation

$R(A, B, C, D)$

Functional dependencies

$A \rightarrow B$
 $B \rightarrow C$
 $C \rightarrow D$

Step 1: Find candidate key

Try A :

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow D$

So:

$A^+ = A, B, C, D$

Thus A is a candidate key.

Step 2: Prime and non-prime attributes

Attribute	Type
A	Prime
B	Non-prime
C	Non-prime
D	Non-prime

Step 3: Check BCNF

For BCNF, every FD must have a superkey on the left.

Check $A \rightarrow B$

- A is a superkey
- OK

Step 1: Find candidate key

Try A :

- $A \rightarrow B$
- $B \rightarrow C$
- Now A and C together give AC
- $AC \rightarrow D$
- $D \rightarrow E$

So:

$A^+ = A, B, C, D, E$

Thus A is a candidate key.

Step 2: Prime and non-prime attributes

Attribute	Type
A	Prime
B	Non-prime
C	Non-prime
D	Non-prime
E	Non-prime

Step 3: Check 3NF violations

FD 1: $A \rightarrow B$

- A is a superkey
- Allowed

FD 2: $B \rightarrow C$

- B is not a superkey
- C is non-prime
- Violates 3NF

FD 3: $AC \rightarrow D$

- AC is a superkey because it contains key A
- Allowed

Check $B \rightarrow C$

- B is not a superkey
- Violates BCNF

Check $C \rightarrow D$

- C is not a superkey
 - Violates BCNF
- So relation is not in BCNF.

Step 4: Decompose using violating FD $B \rightarrow C$

Split into:

Relation	Attributes
R_1	(B, C)
R_2	(A, B, D)

Step 5: Check $R_2(A, B, D)$

FDs in R_2 :

- $A \rightarrow B$
- $B \rightarrow C$ is gone because C is not here

Now only relevant FD is

$A \rightarrow B$

Candidate key for R_2 is A . D ? Let us check.

$A \rightarrow B$, but no FD gives D .

So AB determines A, B, D .

Thus AB is a candidate key in R_2 .

Now check BCNF:

- $A \rightarrow B$
- A is not a superkey of R_2 because A alone does not determine D
- So R_2 still violates BCNF.

Step 6: Decompose $R_2(A, B, D)$ using $A \rightarrow B$

Split into:

$D \rightarrow E$

Rélation	Attributs
R3	(A, B)
R4	(A, D)

Final BCNF decomposition

Rélation	Attributs
R1	(A, C)
R3	(A, B)
R4	(A, D)

Final answer for Problem 3

Result	Value
Candidate key	A
BCNF violations	$B \rightarrow C, C \rightarrow D$. Later $A \rightarrow B$ in a subrelation
Final BCNF decomposition	$R_1(A, C), R_3(A, B), R_4(A, D)$

9) How to decide which normal form to apply

If the question says 3NF

Only check

- partial dependency
- composite key
- non-prime attributes

If the question says 3NF

Check:

- candidate key
- prime attributes
- transitive dependency
- whether RHS is prime or LHS is superkey

If the question says BCNF

Check:

Attribute set	Closure	Key?
CD	C, D, E	No
ACD	A, B, C, D, E	Yes
BCD	A, B, C, D, E	Yes

So the candidate keys are:

ACD, BCD

Step 2: Prime and non-prime attributes

Attribute	Type
A	Prime
B	Prime
C	Prime
D	Prime
E	Non-prime

Important point: only ϵ is non-prime.

2) FD-by-FD table for 2NF, 3NF, and BCNF

Legend:

- Right = allowed in that normal form
- Wrong = violates that normal form

FD	2NF	3NF	BCNF
$A \rightarrow B$	Right: B is prime, so this is not a 2NF problem.	Right: RHS B is prime.	Wrong: A is not a superkey.
$B \rightarrow C$	Right: C is prime, so no 2NF violation.	Right: RHS C is prime.	Wrong: B is not a superkey.
$CD \rightarrow E$	Wrong: CD is part of the candidate keys, so CD and BCD are not a problem. This is a partial dependency.	Wrong: CD is not a prime.	Wrong: CD is not a superkey.
$B \rightarrow A$	Right: A is prime, so no 2NF violation.	Right: RHS A is prime.	Wrong: B is not a superkey.

- every FD
- whether determinant is a superkey
- decompose repeatedly until satisfied

10) Very simple decision table

What you see	What it usually means
Part of key determines non-prime	2NF violation
Non-key determines non-key	3NF violation
Determinant is not a superkey	BCNF violation
Prime attribute on RHS	May still be allowed in 3NF
Superkey on LHS	Always safe

11) Best practical method in exams

When you get a decomposition question, use this exact method:

Step-by-step exam method

- Write all FDs
- Find candidate key using closure
- Mark prime and non-prime attributes
- Check 2NF first if asked
- Check for transitive dependency if asked for 3NF
- Check determinant superkey condition if asked for BCNF
- Decompose using the violating FD
- Verify lossless join
- State final relations clearly

12) Final compact summary

Normal form	Main rule	What to decompose on
2NF	No partial dependency	Part of composite key causing dependency
3NF	No transitive dependency	Non-key to non-key dependency
BCNF	Every determinant must be a superkey	Any FD violating superkey rule

3) What this table means

For 2NF

Only this rule matters:

- A part of a composite candidate key must not determine a non-prime attribute

So only:

$$CD \rightarrow E$$

breaks 2NF.

Why the other FDs do not break 2NF:

- $A \rightarrow B$ is okay because B is prime
- $B \rightarrow C$ is okay because C is prime
- $B \rightarrow A$ is okay because A is prime
- $B \rightarrow A$ is okay because A is prime

For 3NF

A dependency is okay if either:

- the left side is a superkey, or
- the right side is prime

So:

- $A \rightarrow B$ is okay because B is prime
- $B \rightarrow C$ is okay because C is prime
- $B \rightarrow A$ is okay because A is prime
- $CD \rightarrow E$ fails because CD is not a superkey and ϵ is non-prime

For BCNF

A dependency is okay only if:

- the left side is a superkey

So:

- $A \rightarrow B$ fails
- $B \rightarrow C$ fails
- $B \rightarrow A$ fails
- $CD \rightarrow E$ fails

That is why BCNF is stricter than 3NF.

4) The right way to decompose

This is the part that usually causes confusion.

If you want, I can next give you a single master table of all FD patterns like $AB \rightarrow C, A \rightarrow BC, B \rightarrow A, CD \rightarrow E$ and tell you exactly whether each one is allowed in 2NF, 3NF, or BCNF.

⊗ ↯ ⚡ ...

⓪OpenAI Platform S5 for an example if I have the example like a b b c c and also other b's so I want to know that what are the steps to solve so we know to find the c's, p and np and then after finding it make a table of each of the f's like columns are the r's and rows are the f's and then now use the wrong right and in the bracket include the details why it actually failed and what caused it violation and after the table we will be knowing that the r's of all of them in the table.

know solve the same problem for the 2.3 and bonfs

Show more ▼

I'm mapping your example into a full "FD-by-FD" checklist first, then I'll show the clean decomposition path and the common mistake that causes confusion.

Thought for 42s >

Absolutely! Let us use your exact example and make it very clear.

Example relation

$$R(A, B, C, D, E)$$

Functional dependencies:

$$A \rightarrow B, B \rightarrow C, CD \rightarrow E, B \rightarrow A$$

1) First find the candidate keys, prime attributes, and non-prime attributes

Step 1: Check closures

Attribute set	Closure	Key?
A	A, B, C	No
B	A, B, C	No

Good approach

- Find candidate keys
- Mark prime and non-prime attributes
- Decide the target normal form
- Look only for FDs that violate that target normal form
- Split using the violating FD only
- Recheck the projected FDs in the new relations
- Repeat until all relations satisfy the target normal form

Wrong approach

- Splitting every FD blindly
- Decomposing FDs that are already allowed in the target NF
- Forgetting to recheck the new relations after each split
- Using the original FDs unchanged inside every smaller relation

The key idea is this:

When you decompose, the FD set changes inside each new relation. You must project the FDs onto the new relation and check again.

5) Decomposition for 2NF

Only $CD \rightarrow E$ violates 2NF.

So split it off.

$$R_1(C, D, E) \\ R_2(A, B, C, D)$$

Check

- $R_1(C, D, E)$ is in 2NF because CD is its key and E depends on the whole key
- $R_2(A, B, C, D)$ is in 2NF because there is no non-prime attribute left

Final 2NF decomposition

Rélation	Attributs
R1	(C, D, E)
R2	(A, B, C, D)

6) Decomposition for 3NF

A single and neat 3NF answer for this example is the same split:

$$R_1(C, D, E), \quad R_2(A, B, C, D)$$

Why this works

- In $\pi_1(C,D,E)$, $CD \rightarrow E$ is fine because CD is the key
- In $\pi_2(A,B,C,D)$, the dependences are:
 - $A \rightarrow B$
 - $B \rightarrow C$
 - $B \rightarrow A$

Here A , B , and C are all prime in π_2 , so these are allowed in 3NF.

Final 3NF decomposition

Relation	Attributes
R1	(C, D, E)
R2	(A, B, C, D)

7) Decomposition for BCNF

Now we must fix every FD whose left side is not a superkey.

A valid BCNF decomposition sequence is:

Step 1: Use $CD \rightarrow E$

Step 2: In $R_2(A,B,C,D)$, use $B \rightarrow C$

Step 3: In $R_{22}(A,B,D)$, use $A \rightarrow B$

Final BCNF relations

Relation	Attributes
R1	(C, D, E)
R21	(B, C)
R23	(A, B)
R24	(A, D)

Check each one

- CDE : CD is a key, so BCNF
- BC : B is a key, so BCNF

- BCNF decomposition:
 - $\pi_1(A, B)$
 - $\pi_2(B, C)$
 - $\pi_3(C, D)$

If you want, I can turn all of this into a single neat exam sheet with just tables and final decomposition rules, so it is easy to revise in one glance.

- AB : A and B are keys here because $A \rightarrow B$ and $B \rightarrow A$, so BCNF
- AD : no nontrivial FD, so BCNF

8) Why this is the correct decomposition style

You asked the most important conceptual question: why do we split only the violated FD, and what makes that the good approach?

Because decomposition is driven by the target NF

- For 2NF: fix only partial dependency problems
- For 3NF: fix transitive dependency problems or use synthesis
- For BCNF: fix every FD whose determinant is not a superkey

Why not split everything?

Because many FDs are already acceptable in the target NF.

Example:

- In 3NF, $A \rightarrow b$ is okay here because b is prime
- In 2NF, $A \rightarrow b$ is okay here because b is prime

So if you split that FD too early, you create unnecessary tables.

Why rechecking is necessary

After one split, a new relation may still contain a violating FD.

That is exactly why BCNF is often solved in stages:

- decompose once
- project the FDs
- check again
- decompose again if needed

9) Ultra-short decision rule

Normal form	What to check first
2NF	Partial dependency of non-prime attributes on part of a key
3NF	Non-superkey to non-prime dependency
BCNF	Every determinant must be a superkey

10) Three rough practice problems with answers only

Problem 1

Relation:

$R(A, B, C, D, E)$

FDs:

$AB \rightarrow C, A \rightarrow D, D \rightarrow E$

Answer

- Candidate key: AB
- Prime attributes: A, B
- 2NF decomposition: $\pi_1(A, D)$, $\pi_2(A, B, C, E)$
- 3NF/BCNF need further splitting because $B \rightarrow E$ still causes violation in the second relation

Problem 2

Relation:

$R(A, B, C, D, E)$

FDs:

$A \rightarrow B, B \rightarrow C, AC \rightarrow D, D \rightarrow E$

Answer

- Candidate key: A
- Prime attributes: A
- Final 3NF decomposition:
 - $\pi_1(A, B)$
 - $\pi_2(B, C)$
 - $\pi_3(A, C, D)$
 - $\pi_4(D, E)$

Problem 3

Relation:

$R(A, B, C, D)$

FDs:

$A \rightarrow B, B \rightarrow C, C \rightarrow A, C \rightarrow D$

Answer

- Candidate keys: A, B, C
- Prime attributes: A, B, C